Design idempotent post api

31 May 2025 22:33

Design Idempotent Post API

Difference b/w Idempotency & Concurrency

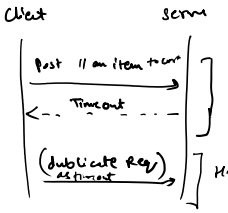
Concurrency is when multiple Request for single Resource eg Book my show getting same seat
Idempotency helps to take care of duplicate Request
It enables clients do safely retry an operation without worrying about the side-effect of operation can cause

eg – Add an item into Cart
– making an Payment

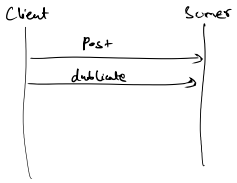
→ Get, Put, Delete are Idempotent in nature as these objects doesn't cause side effect on DB
→ Post is not Idempotent & we need to make it Idempotent

There are two types of Side Affects

Sequential



Parllel

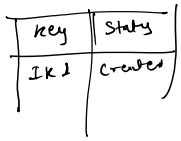


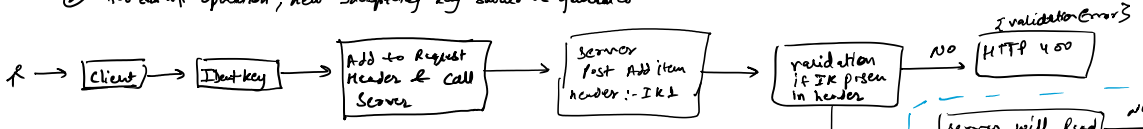
Same time, So we need to handle this as only one Request should be processed

Approch for Idempotency handling

Using Idempotency key, it is unique (UUID) universal unique ID

There are Agreement with client
① client should generate idempotency key
② for each diffnt operation, new Idempotency key should be generated





IDK Life cycles

Created/Aquired
Consumed / Claimed [if claimed it cannot be reused]

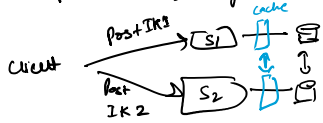
Parllel Request will be handled by mutex or mutal Exclusion

Critical Section

this can be handled by Synchronization or mutex as single thread can access this section
or
we say mutal exclusion

mutex → aquire / Release
Synchronization → lock / unlock

Q What if same request goes to multiple servers?



Ans We can handle using Cache as it is much faster than DB [milliseconds], cache synchronization very faster
So we put lock in cache